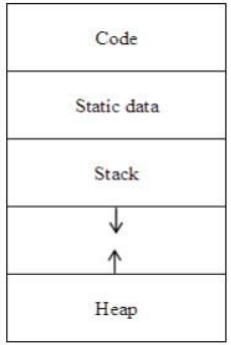
Storage Organization

Storage Organization

- Executing target program runs in its own logical address space in which each program value has a location.
- Management and organization of this logical address space is shared between the compiler, operating system and target machine.
- Operating system maps the logical address into physical addresses, which are usually spread throughout memory.



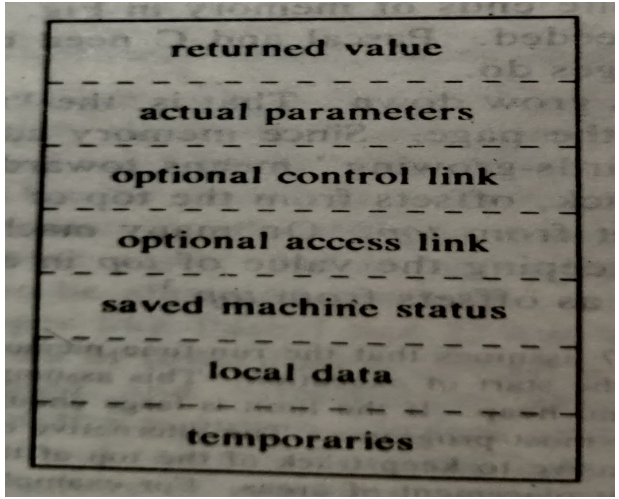
- Run-time storage comes in blocks, where a byte is the smallest unit of memory.
- Four bytes form a machine word.
- Multibyte objects are bytes and given the address of first byte.
- Storage layout for data objects is strongly influenced by the addressing constraints of the target machine.
- Character array of length 10 needs only enough bytes to hold 10 characters, a compiler may allocate 12 bytes to get alignment, leaving 2 bytes unused.
- Unused space due to alignment considerations is referred to as padding.
- Size of some program objects may be known at run time and may be placed in an area called static.
- Dynamic areas used to maximize the utilization of space at run time are stack and heap.

Activation records

- Procedure calls and returns are usually managed by a run time stack called the control stack.
- Each live activation has an activation record on the control stack, with the root of the activation tree at the bottom, the latter activation has its record at the top of the stack.
- Contents of the activation record vary with the language being implemented.
- Temporary values such as those arising from the evaluation of expressions.

- Local data belonging to the procedure whose activation record this is.
- Saved machine status, with information about the state of the machine just before the call to procedures.
- An access link may be needed to locate data needed by the called procedure but found elsewhere.
- Control link pointing to the activation record of the caller.

- Space for the return value of the called functions, if any. Again, n return a value, and if one does, we may prefer to place that value i efficiency.
- Actual parameters used by the calling procedure.
- These are not placed in record but rather in registers, when possible, for greater efficiency.



General Activation record

- **Temporary values** arise in the evolution of expressions, are stored in the field for temporaries.
- Field for **local data** holds data that's local to an execution of a procedure.
- Field for **saved machine status** holds information about the state of the machine just before the procedure is called.
  - ★ Information includes values of the program counter and machine registers that have to be restored when control returns from the procedure.

- **Optional access link** is used to refer to nonlocal data held in other activation records.
- **Optional control link** points to the activation record of the caller.
- Field for **actual parameters** is used by the calling procedure to supply parameters to the called procedure.
- Field for the **returned value** is used by the called procedure to return a value to the calling procedure.

STORAGE ALLOCATION STRATEGIES

Different storage allocation strategies are :

1. Static allocation - lays out storage for all data objects at compile time
2. Stack allocation - manages the run-time storage as a stack.
3. Heap allocation - allocates and deallocates storage as needed at runtime from a data area known as heap.

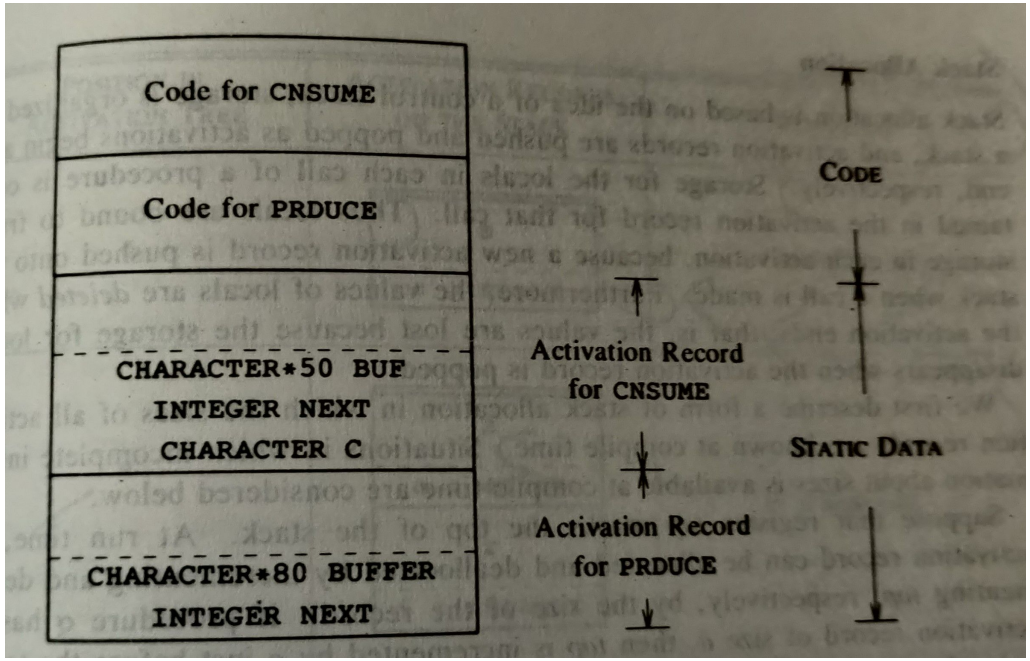
STATIC ALLOCATION

- Static allocation, names are bound to storage as the program is compiled, so there is no need for a run-time support package.
- Bindings do not change at run-time, everytime a procedure is activated, its names are bound to the same storage locations.
- Values of local names are retained across activations of a procedure.
- When control returns to a procedure the values of the locals are the same as they were when control left the last time.
- From the type of a name, the compiler decides the amount of storage for the name and decides where the activation records go.
- At compile time, we can fill in the addresses at which the target code can find the data it operates on.

STATIC ALLOCATION

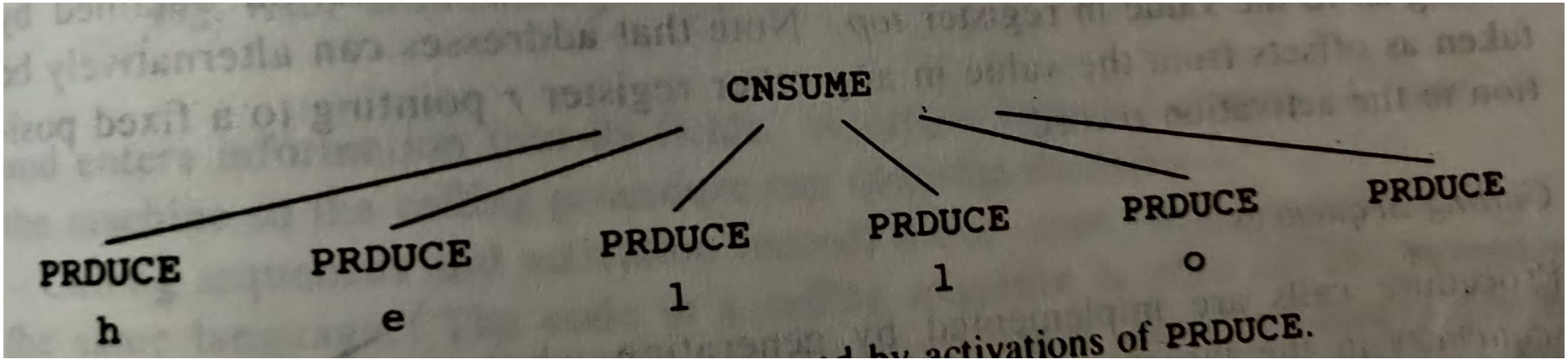
```
(1)      PROGRAM CNSUME
(2)         CHARACTER * 50 BUF
(3)         INTEGER NEXT
(4)         CHARACTER C, PRDUCE
(5)         DATA NEXT /1/, BUF /' '/
(6)  6         C = PRDUCE()
(7)            BUF(NEXT:NEXT) = C
(8)            NEXT = NEXT + 1
(9)            IF ( C .NE. ' ' ) GOTO 6
(10)           WRITE (*,'(A)') BUF
(11)           END
(12)     CHARACTER FUNCTION PRDUCE()
(13)        CHARACTER * 80 BUFFER
(14)        INTEGER NEXT
(15)        SAVE BUFFER, NEXT
(16)        DATA NEXT /81/
(17)           IF ( NEXT .GT. 80 ) THEN
(18)              READ  (*,'(A)') BUFFER
(19)              NEXT = 1
(20)           END IF
(21)           PRDUCE = BUFFER(NEXT:NEXT)
(22)           NEXT = NEXT+1
(23)           END
```

FORTRAN 77 program



Static storage for local identifiers of FORTRAN 77 program

STATIC ALLOCATION



Characters returned by activations of PRDUCE

STACK ALLOCATION OF SPACE

- All compilers for languages that use procedures, functions or methods as units of user-defined actions manage at least part of their run-time memory as a stack.
- Each time a procedure is called , space for its local variables is pushed onto a stack, and when the procedure terminates, that space is popped off the stack.

Calling sequences

- Procedures called are implemented in what is called as calling sequence, which consists of code that allocates an activation record on the stack and enters information into its fields.
- A return sequence is similar to code to restore the state of machine so the calling procedure can continue its execution after the call.
- The code in calling sequence is often divided between the calling procedure (caller) and the procedure it calls (callee).

- ❑ When designing calling sequences and the layout of activation records, the following principles are helpful:
- ★ Values communicated between caller and callee are generally placed at the beginning of the callee's activation record, so they are as close as possible to the caller's activation record.
- ★ Fixed length items are generally placed in the middle. Such i the control link, the accesslink, and the machine status fields

- ★ Items whose size may not be known early enough are placed at the end of the activation record.
- ★ Most common example is dynamically sized array, where the value of one of the callee's parameters determines the length of the array.
- ★ Common approach is to have it point to the end of fixed-length fields in the activation record.
- ★ Fixed-length data can then be accessed by fixed offsets, known to the intermediate-code generator, relative to the top-of-stack pointer.

- The calling sequence and its division between caller and callee are as follows:
  - ★ The caller evaluates the actual parameters.
  - ★ The caller stores a return address and the old value of top_sp into the callee's activation record.
  - ★ The caller then increments the top_sp to the respective positions.
  - ★ The callee saves the register values and other status information.
  - ★ The callee initializes its local data and begins execution.

- A suitable, corresponding return sequence is:
  - ★ The callee places the return value next to the parameters.
  - ★ Using the information in the machine-status field, the callee restores top sp and other registers, and then branches to the return address that the caller placed in the status field.
  - ★ Although top_sp has been decremented, the caller knows where the return value is, relative to the current value of top_sp; the caller therefore may use that value.

Variable length data on stack

- Run-time memory management system must deal frequently with the allocation of space for objects, the sizes of which are not known at the compile time, but which are local to a procedure and thus may be allocated on the stack.
- Reason to prefer placing objects on the stack is that we avoid the expense of garbage collecting their space.

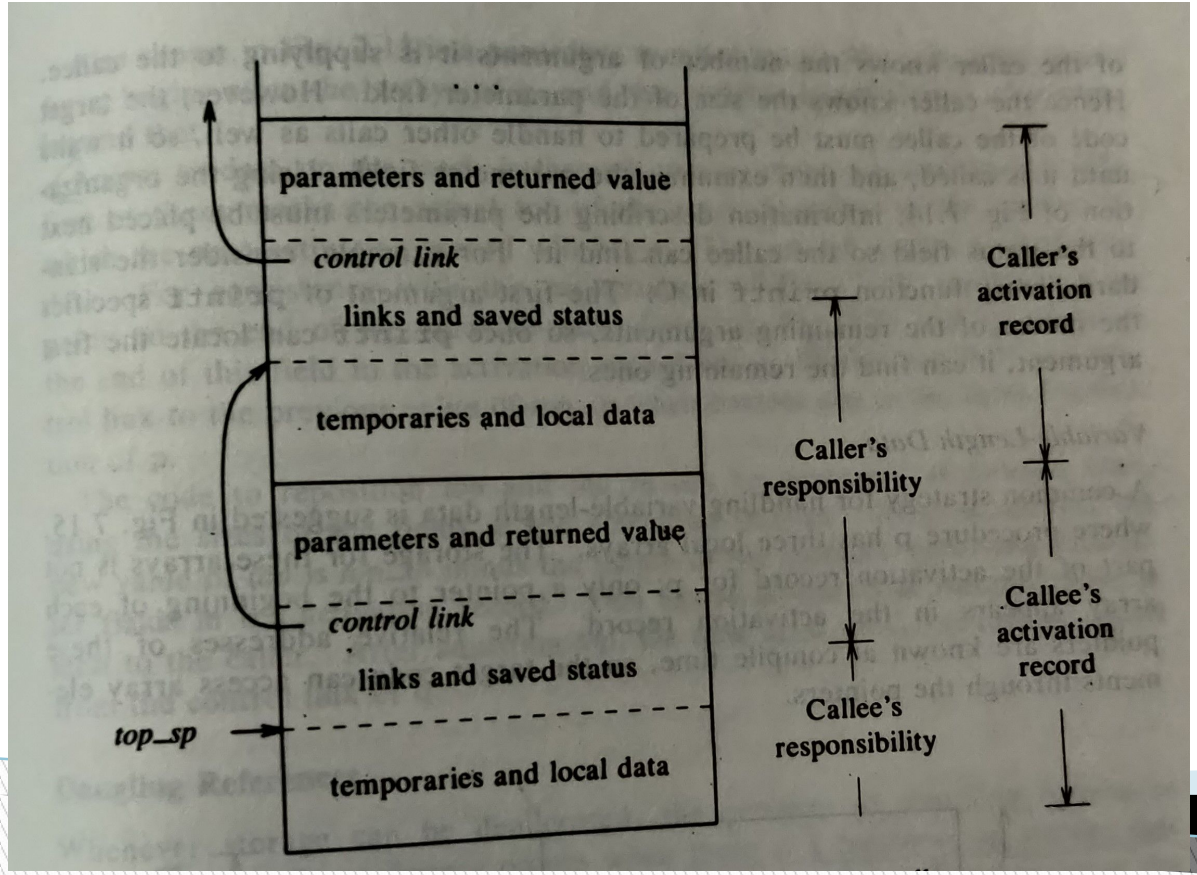
- Same scheme works for objects of any type if they are local to the procedure called and have a size that depends on the parameters of the call.

Dangling Reference

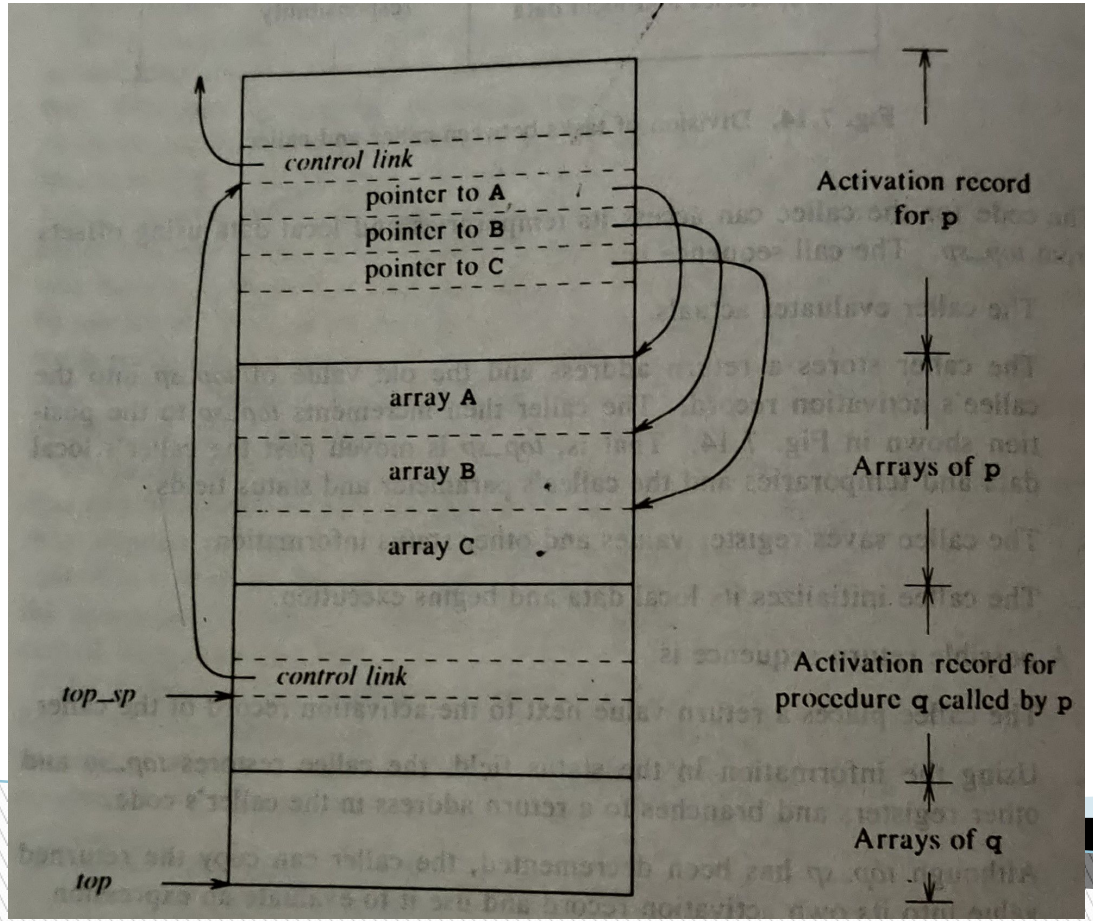
- Dangling reference occurs when there is a reference to storage that has been deallocated.

Position in Activation Tree	Activation Records on the Stack	Remarks
s	s a : array	Frame for s
s r	s a : array r i : integer	r is activated
s r q(1,9)	s a : array q(1,9) i : integer	Frame for r has been popped and q(1,9) pushed
s r q(1,9) p(1,9) q(1,3) p(1,3) q(1,0)	s a : array q(1,9) i : integer q(1,3) i : integer	Control has just returned to q(1,3)

Downward- growing stack allocation of activation records



Division of task between caller and callee



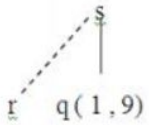
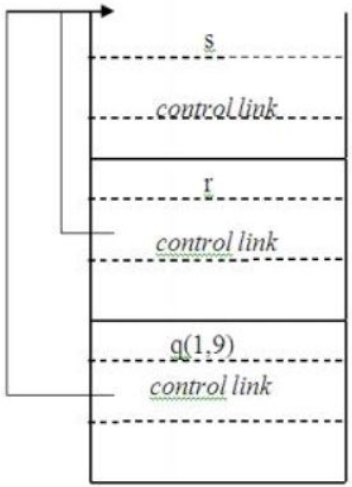
Access to dynamically allocated arrays

HEAP ALLOCATION

- Stack allocation strategy cannot be used if either of the following is possible :

1. The values of local names must be retained when an activation ends.
2. A called activation outlives the caller.

- Heap allocation parcels out pieces of contiguous storage, as needed for activation records or other objects.
- Pieces may be deallocated in any order, so over the time the heap will consist of alternate areas that are free and in use.

Position in the activation tree	Activation records in the heap	Remarks
		Retained activation record for r

Records for live activations need not be adjacent in heap

Thank
you!